

M11 / PROTECTED ANSWERS

M11 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

M11 protected revision test answers 3

DE11-01 - choose for the workload 3

DE11-02 - solve the metadata decisions 3

DE11-03 - output columns are not all read columns 3

DE11-04 - two different file-size problems 4

DE11-05 - choose partitioning from evidence 4

DE11-06 - make the architecture concrete 4

DE11-07 - follow active files through a commit 4

DE11-08 - specify this fixture’s merge contract 5

DE11-09 - assess consumer compatibility 5

DE11-10 - write a recovery-aware runbook 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

M11 protected revision test answers

Open this after saving your own attempt. These answers solve the printed questions. Simulator fixtures and real file metadata are identified separately.

DE11-01 - choose for the workload

For frequent point lookup of one customer, use a row-oriented database with a suitable customer key/index: the workload wants a small record quickly and may need updates and transactions. A CSV scan would inspect many unrelated records unless another index were built. For a nightly sum by region across millions of orders, Parquet is a useful typed columnar representation: a reader can select region and amount, benefit from compression, and skip some row groups when suitable filter statistics exist. Parquet itself is a file format, not a complete database service. For a nontechnical partner, a documented CSV can be the easiest exchange format, but agree delimiter, encoding, date format and IDs-as-text rules. No format is best for all three jobs.

DE11-02 - solve the metadata decisions

The supplied JSON is a teaching description, not metadata read from a real Parquet file. RG0 has dates September 1-2 and amount min/max 80/200. RG1 has dates September 3-4 and amount min/max 90/190. For `order_date >= September 3`, skip RG0 because its maximum date is September 2; RG1 remains a candidate. For `amount > 195`, skip RG1 because its maximum is 190; keep RG0 because its maximum is 200. Keeping a group does not mean every row in it matches. For `region = East`, neither group can be eliminated using this fixture, because it has no region statistics. Guessing region from the file name or another column would be unsupported.

DE11-03 - output columns are not all read columns

Query A returns `order_id` and `amount` where `order_date >= September 3`. Output columns are `order_id/amount`; `order_date` is additionally needed to evaluate the filter. Only RG1 is a candidate from the supplied statistics. Query B returns `order_id` where `amount > 195`. Output is `order_id`, filter input is `amount`, and only RG0 is a candidate. Query C returns `order_id/amount` where `region = East`. The filter needs `region` and both groups remain candidates. To prove real pruning, inspect the engine's physical scan plan/scan metrics and the actual file's column statistics. A pushed filter in a plan is helpful, but reader metrics such as row groups or bytes actually scanned give stronger execution evidence. Faster elapsed time alone can also come from cache or unrelated machine activity. The teaching JSON can prove a skipping opportunity, not that a native reader used it.

DE11-04 - two different file-size problems

Forty files of 3 MB total 120 MB. If they belong to the same compatible partition and schema, combining them into fewer files can reduce per-file open and metadata overhead. Four files of 600 MB total 2400 MB; those are already much larger. Do not automatically combine them too. First inspect row groups, parallelism, memory, partition boundaries and the consuming engine's behavior. One huge output may reduce parallel work or make an individual task expensive. There is no universal ideal size. Reconcile rows, keys and business totals before and after any physical rewrite.

DE11-05 - choose partitioning from evidence

For a 2 TB order table queried mostly by recent date ranges, date partitioning is a plausible starting point. Ask how many days the data spans, rows/bytes per day, late-update frequency, typical query filters and target file sizes. Two terabytes over two days differs from two terabytes over ten years. Region works only if the number and distribution of regions and query patterns justify it. `customer_id` and especially `order_id` may create enormous numbers of tiny partitions; count distinct values and estimate bytes per partition before choosing. No partitioning can be preferable when layout statistics and engine optimizations already serve the workload well. Test a representative query and write/update pattern rather than choosing by table size alone.

DE11-06 - make the architecture concrete

Lake option: preserve event files in object storage and use a separate engine/catalog to curate and query them; define who enforces schema, key and publication rules. Warehouse option: keep raw archives in object storage, load validated finance tables into managed analytical storage, and give BI users governed SQL access. Lakehouse option: retain raw files and maintain trusted tables over object storage with a transaction-log table format, then expose curated SQL tables to BI. In all three, the finance amount and reconciliation rules still belong to the business contract. Shared storage does not automatically grant all users raw access or make every table trustworthy.

DE11-07 - follow active files through a commit

Imagine version 0 lists active files A and B. A writer prepares replacement file C and wants version 1 to contain A and C, with B removed from the active snapshot. Until the version-1 log commit succeeds, a Delta reader sees A and B. After it succeeds, a new reader can see A and C. B may still exist physically for historical reads. If the writer prepares D for version 2 but fails before committing, readers continue to use the last committed snapshot; the presence of D does not make it current. This is the atomic visibility boundary of a table commit.

DE11-08 - specify this fixture's merge contract

The target grain is one current row per `order_id` and `order_id` is the match key. The supplied changes file contains two known keys and one new key. Reject duplicate source keys before mutation; for this introductory ordered-batch fixture, each accepted change replaces the matching current row. There is no separate `updated_at` field here, so do not claim arbitrary out-of-order correction handling. Later modules add explicit version precedence. O1003 becomes 215, O1005 becomes 95, and O1009 is inserted at 130. Starting total is 1100. The two updates add a net 20 and the insert adds 130, giving 1250. There are 9 rows and 9 distinct order IDs. Replay sets the same values and leaves 9/1250 unchanged. Appending the changes would create duplicate current keys. Adding the incoming amount to the old amount would multiply revenue on replay. The duplicate-source fixture must fail before target mutation.

DE11-09 - assess consumer compatibility

Adding nullable `source_system` is usually a controlled additive change: existing rows receive null and consumers must tolerate it. Renaming `customer_id` needs consumer migration because queries and joins using the old name break. Converting int to double may be technically supported in a specific engine, but exact integer/identifier behavior and large-value precision can change; treat it as a reviewed type migration rather than harmless widening. Changing string to struct changes the access pattern and contract fundamentally and needs an explicit migration or rebuilt representation. Dropping region breaks any report that groups or filters by it, so migrate consumers first. Schema evolution should record approval, expected old-row values and compatibility tests, not merely enable an automatic option.

DE11-10 - write a recovery-aware runbook

Record the current table version and required historical recovery window. Identify the person/process owning retention and the longest reader/stream delay you must support. Inspect history and confirm the desired historical version is readable before maintenance. Run VACUUM in dry-run mode with the approved retention and inspect candidate files. Keep safety checks enabled. Only perform deletion when the policy allows it, then validate current keys, counts, totals and the historical reads the policy promises. A course simulator listing files with `deleted=false` is a plan, not a real VACUUM execution. For this module, Bronze preserves the received order bytes and acquisition identity. Silver validates typed fields, source disposition and current order keys. Gold serves reconciled business aggregates. After the fixture merge, current state has 9 keys/1250; version 0 has 8 keys/1100 while its required files remain retained. A successful physical compaction must preserve those logical current controls. Do not run aggressive cleanup to solve a bad data transformation.